

A transformação WebAssembly $\rightarrow$ LLVM-IR no Map2Check

Formalização da cadeia de *lifting*, e o que ela demonstravelmente preserva e perde

Projeto Map2Check

4 de setembro de 2026

Resumo

Formalizamos a cadeia de transformações que leva um binário `.wasm` até o programa instrumentado que o Map2Check analisa, decompondo-a em cinco funções e enunciando explicitamente as duas hipóteses de correção sobre as quais ela repousa. Dois resultados decorrem da formalização. A cadeia é *sound* para as classes de *trap* cujo teste o código gerado efetivamente contém, um subconjunto próprio das dez do WebAssembly, e dependente do módulo, pois o teste de limites da memória linear é elidido na configuração de compilação empregada, e isso já vale na implementação atual, sem vir sendo reivindicado. E a cadeia é incompleta para segurança de memória intra-*sandbox*, precisamente a classe de vulnerabilidade declarada como alvo do projeto, por uma razão estrutural: a memória linear chega ao verificador como uma única alocação de *heap*, de modo que a sobrescrita de um objeto vizinho não é observável por predicado algum em jogo. Damos o contraexemplo por construção, verificado por execução, e derivamos do enunciado o requisito exato que uma modelagem de memória linear teria de satisfazer, requisito que, como se verá, está parcialmente implementado e inoperante.

Sumário

1	Objetivo e escopo	2
2	Sumário dos resultados	3
3	Notação	4
4	Passo 0, o objeto de entrada	4
4.1	A memória linear	4
5	Passo 1, τ: o tradutor wasm2c	5
5.1	Representação do estado	6
5.2	A condição lateral torna-se código	6
5.3	Ressalva decisiva: o <i>bounds check</i> é condicional	6
5.4	Correção de τ	7
6	Passo 2, κ: Clang para LLVM-IR	8
7	Passo 3, ϵ: extração do ponto de entrada	8

8	Passo 4, ω: o <i>harness</i>	9
8.1	Um defeito de aridade no <i>wrapper</i>	9
8.2	O que ω quantifica: nada	9
9	Passo 5, π: a instrumentação do Map2Check	10
9.1	O <i>runtime</i> efetivo: os <i>stubs</i> do Map2Check	10
9.2	A incompatibilidade entre compilação e <i>runtime</i>	11
9.3	O mapa de objetos existe, e não dispara	11
10	Proposição 1: o que a cadeia preserva	12
10.1	Leitura correta do resultado	12
11	Proposição 2: o que a cadeia não preserva	13
12	Consequência para a validação existente	14
13	O que a modelagem de memória linear tem de acrescentar	15
14	Resumo da cadeia	15
15	Recomendações, em ordem	15
A	Protocolo de verificação	16
A.1	A elisão do teste de limites	17
A.2	Determinação de $\mathcal{T}_{\text{emit}}$	17
A.3	O contraexemplo	17
A.4	A aridade do <i>wrapper</i>	17
A.5	O que não foi verificado	18
B	Glossário de símbolos e termos	18
B.1	Notação de base	18
B.2	O módulo e seu estado de execução	19
B.3	A memória linear e o acesso	20
B.4	As funções da cadeia	21
B.5	Semântica denotacional e alcançabilidade	21
B.6	A instrumentação e os predicados de segurança	22
B.7	Três distinções que o glossário não deve deixar passar	22

1 Objetivo e escopo

Este documento descreve, passo a passo e por meio de uma formalização explícita, a cadeia de transformações que leva um binário `.wasm` até o programa instrumentado analisado pelo Map2Check.

O objetivo não é apenas documentar a implementação, mas tornar **preciso** o que a cadeia preserva e o que ela perde. Essa distinção é fundamental para determinar quais resultados experimentais podem ser atribuídos ao Map2Check e quais não podem. Duas proposições emergem dessa formalização, e ambas delimitam o que pode ser afirmado sobre a frente WASM.

A semântica de referência do WebAssembly é a de Haas et al., *Bringing the Web up to Speed with WebAssembly* (PLDI 2017), e a especificação normativa dela derivada. O tradutor é o `wasm2c` do WABT 1.0.41. Todas as afirmações sobre o código gerado foram

verificadas na saída real do WABT na imagem `ghcr.io/hbgit/map2check-dev`, não inferidas da documentação.

Referências de implementação: `modules/frontend/wasm_lifter.{hpp,cpp}`, `modules/frontend/caller.cpp` (função `generateWasmWrapperStatic`) e `modules/frontend/map2check.cpp` (bloco `args.wasmMode`).

2 Sumário dos resultados

Para o leitor que precisa apenas da conclusão, antes do desenvolvimento.

Proposição 1, a cadeia é *sound* para os *traps* cujo teste o código gerado contém. O `wasm2c` traduz as condições laterais das regras de redução em testes explícitos, e o *stub* de `wasm_rt_trap` do Map2Check encaminha o resultado a `map2check_error()`, o mesmo sinal que o modo de alcançabilidade já consome. Divisão por zero, *overflow* de divisão, *unreachable* e as demais classes cujo teste é emitido incondicionalmente **são detectáveis hoje**, sem trabalho adicional. Este resultado existe e não estava sendo alegado.

Duas ressalvas o delimitam, e nenhuma é secundária. Primeira: para memórias lineares de 32 bits com página padrão, e na configuração de compilação que o Map2Check emprega, o teste de limites **não é emitido**, o WABT o substitui por páginas-guarda (Seção 5.3). Segunda: a proposição é de *soundness*, isto é, de ausência de falso positivo; detectabilidade é a recíproca, e depende ainda de o *harness* alcançar o teste (Seção 8.2).

Uma incompatibilidade entre compilação e *runtime*. O código gerado omite a verificação de limites porque supõe páginas-guarda, enquanto o *runtime* que o Map2Check efetivamente liga aloca a memória linear com `calloc`, que não as tem (Seção 9.2). O acesso fora dos limites não é detido por teste emitido nem por hardware.

Proposição 2, a cadeia é incompleta para segurança de memória intra-*sandbox*. Esta é justamente a classe de vulnerabilidade declarada como alvo principal do projeto. A memória linear chega ao verificador como **uma única alocação**, de modo que a sobrescrita de um objeto vizinho *dentro* dela não é observável pelo predicado de segurança do Map2Check. A demonstração é por construção, o contraexemplo tem quatro linhas de C, e foi **confirmada por execução**. A causa é estrutural, não um defeito de implementação.

O mapa de objetos existe, e não dispara. O `MemoryTrackPass` tem um caminho WASM dedicado que rastreia alocações *por deslocamento dentro da memória linear*, em substância, o mapa de objetos que a Seção 13 descreve como o que falta. Ele não dispara: o casamento de padrão exige uma forma de instrução que o IR do `wasm2c` não produz (Seção 9.3). Construir esse mapa deixa, portanto, de ser pesquisa e passa a ser depuração.

Consequência imediata. A validação existente em `docs/reports/wasm-juliet-validation.csv` não separa os mecanismos possíveis de detecção, e a Proposição 2 mostra que um deles é impossível no caso intra-memória. Enquanto essa separação não for feita, os dados são consistentes com a hipótese de que **a tradução encontrou os defeitos e o Map2Check apenas reportou o sinal resultante** (Seção 12).

3 Notação

Esta seção fixa apenas a notação de base. **Tudo símbolo empregado no documento, incluindo os introduzidos adiante, está reunido, com leitura em palavras e definição, no glossário do Apêndice B.**

Seja $\mathbb{B} = \{0, \dots, 255\}$ o conjunto dos bytes. Escrevemos $[a, b)$ para o intervalo inteiro semiaberto. Para uma função parcial f , $\text{dom}(f)$ é seu domínio.

O tamanho de página do WebAssembly é a constante

$$P = 65\,536.$$

Os tipos de valor são $\text{valtype} = \{\text{i32}, \text{i64}, \text{f32}, \text{f64}\}$, e Val denota o conjunto dos valores correspondentes.

4 Passo 0, o objeto de entrada

Um módulo WebAssembly é a tupla

$$M = (\text{types}, \text{funcs}, \text{tables}, \text{mems}, \text{globals}, \text{elem}, \text{data}, \text{start}, \text{imports}, \text{exports}).$$

Três componentes importam para o que segue.

- *mems*, que na versão 1.0 do WebAssembly tem **cardinalidade no máximo 1**. Esta é a hipótese estrutural que sustenta todo o resto do documento: existe uma única memória linear por módulo.
- *exports* : $\text{Name} \rightarrow \text{ExternIdx}$, que dá as funções visíveis de fora do módulo.
- *imports*, que dá as funções fornecidas pelo *host*. Em contexto de IoT, é por aqui que passam acesso a sensores, rede e banco de dados.

O **estado de execução** é o par (s, F) , com *store*

$$s = (\text{funcs}, \text{tables}, \text{mems}, \text{globals})$$

e *frame* F contendo os locais e a instância corrente. Uma configuração é $s; F; e^*$, onde e^* é a pilha de instruções, e a semântica é dada por uma relação de redução em passos pequenos

$$s; F; e^* \hookrightarrow s'; F'; e'^*.$$

O termo distinguido **trap** é absorvente: nenhuma regra o reduz.

4.1 A memória linear

Uma instância de memória linear é definida como

$$\mu = (d, \ell), \quad d : [0, \ell) \rightarrow \mathbb{B}, \quad \ell = n \cdot P$$

para algum $n \in \mathbb{N}$.

Isto é: **um vetor plano de bytes, contíguo, endereçado por inteiros de 32 bits**, sem qualquer estrutura interna. Não há noção de objeto, de alocação ou de fronteira entre variáveis. Este fato é central para tudo o que segue e merece ser explicitado: a memória linear não é um espaço de endereçamento que contém objetos distintos; é *um único objeto*.

Os símbolos da instrução de acesso. A regra abaixo emprega seis identificadores que convém fixar antes de lê-la, pois reaparecem em todo o documento:

- t o tipo de valor lido ou escrito, elemento de *valtype*; aparece como prefixo da instrução, em *t.load*.
- N a **largura do acesso em bits**, 8, 16, 32 ou 64. Logo $N/8$ é o número de bytes tocados pela instrução, e é sempre nessa forma que N comparece.
- i o **índice dinâmico**, valor *i32* retirado do topo da pilha de operandos. É a parte do endereço que depende da execução.
- o set** o **deslocamento estático**, constante imediata codificada na própria instrução.
- ea o **endereço efetivo** (*e ective address*), definido como $ea = i + \text{o set}$. É o índice, em bytes, dentro da memória linear, a partir do qual o acesso ocorre. Não é um ponteiro do processo: é um inteiro em $[0, \ell)$.
- c o valor resultante da leitura, do tipo t .

O acesso, portanto, toca os bytes de índice ea até $ea + N/8 - 1$, e a questão de segurança é apenas se esse intervalo cabe em $[0, \ell)$.

A regra de leitura, na forma relevante para a análise, é

$$\frac{ea = i + \text{o set} \quad ea + N/8 \leq \ell}{s; F; (\text{i32.const } i) (t.\text{load } N \text{ o set}) \hookrightarrow s; F; (t.\text{const } c)} \quad \text{LOAD}$$

e, quando a condição lateral falha,

$$\frac{ea + N/8 > \ell}{s; F; (\text{i32.const } i) (t.\text{load } N \text{ o set}) \hookrightarrow s; F; \text{trap}} \quad \text{LOAD-TRAP}$$

A regra de escrita é simétrica: a mesma condição lateral governa *t.store*, que consome um valor adicional da pilha e não devolve resultado. O contraexemplo da Seção 11 é uma escrita, e é a essa regra que ele apela.

É importante observar a forma da condição lateral,

$$ea + N/8 \leq \ell.$$

Essa mesma condição reaparece, na forma de código C, na seção seguinte.

5 Passo 1, τ : o tradutor *wasm2c*

Definimos

$$\tau : \text{Wasm} \longrightarrow C$$

como a função implementada pelo *wasm2c* do WABT 1.0.41, invocada em *WasmLifter::runWasm2c* por

```
| wasm2c <entrada>.wasm -o <saida>.c
```

produzindo o par de arquivos *.c* e *.h*.

5.1 Representação do estado

τ realiza o *store* s como uma **estrutura de instância** I . A memória linear $\mu = (d, \ell)$ torna-se o registro `wasm_rt_memory_t`, cujos campos relevantes são

$$\text{data} \mapsto d, \quad \text{size} \mapsto \ell,$$

com `data` obtido de uma **única** chamada de alocação, em `wasm_rt_allocate_memory`. Registre-se o quantificador: *uma* alocação, para *toda* a memória do módulo. É desse fato que a Proposição 2 decorre.

Cada função f do módulo torna-se uma função C

$$\tau(f) : \text{w2c_}\langle \text{mod} \rangle * \times \overline{\text{Val}} \longrightarrow \text{Val},$$

nomeada segundo o esquema `w2c_<módulo>_<função>`. O primeiro parâmetro é o ponteiro de instância: é assim que o estado global do WASM se torna estado explícito de primeira classe no C.

5.2 A condição lateral torna-se código

Eis o ponto técnico central. τ **compila a condição lateral da regra de redução em uma verificação dinâmica explícita**. No C gerado, verificado na saída real do WABT 1.0.41:

```
#define MEM_ADDR(mem, addr, n) &((mem)->data[addr])

#define RANGE_CHECK(mem, offset, len) \
do { \
    uint64_t res; \
    if (UNLIKELY(add_overflow(offset, len, &res))) \
        TRAP(OOB); \
    if (UNLIKELY(res > (mem)->size)) \
        TRAP(OOB); \
} while (0);

#define TRAP(x) (wasm_rt_trap(WASM_RT_TRAP_##x), 0)
```

Comparando com a regra LOAD: `res > mem->size` é exatamente a negação de $ea + N/8 \leq \ell$, e `TRAP(OOB)` é exatamente `trap`. A tradução é literal, *onde este macro é usado*, ressalva que a Seção 5.3 torna decisiva.

Definindo então o predicado

$$\text{Safe}_{\text{wasm}}(\mu, ea, N) \equiv ea + N/8 \leq \ell,$$

o programa C avalia $\text{Safe}_{\text{wasm}}$ em tempo de execução antes de cada acesso, desviando para `wasm_rt_trap` quando ele é falso, **sob uma das duas estratégias de verificação de limites que o WABT oferece**. A configuração efetivamente empregada pelo Map2Check não é essa, e a subseção seguinte trata do ponto.

5.3 Ressalva decisiva: o *bounds check* é condicional

O `wasm-rt.h` seleciona a estratégia em tempo de compilação: em plataformas de 64 bits com alocação por `mmap`, x86-64, portanto, o padrão é `WASM_RT_MEMCHECK_GUARD_PAGES`; caso contrário, `WASM_RT_MEMCHECK_BOUNDS_CHECK`. O acesso a memórias lineares de 32 bits com página padrão, que é o caso comum, não usa `RANGE_CHECK`, e sim o macro acelerado `MEMCHECK_DEFAULT32`:

```

#if WASM_RT_MEMCHECK_GUARD_PAGES
#define MEMCHECK_DEFAULT32(mem, a, t) WASM_RT_CHECK_BASE(mem);
#else
#define MEMCHECK_DEFAULT32(mem, a, t) \
    WASM_RT_CHECK_BASE(mem); \
    if (UNLIKELY(a + (uint64_t)sizeof(t) > mem->size)) \
        TRAP(OOB);
#endif

```

Sob `GUARD_PAGES` resta apenas `WASM_RT_CHECK_BASE`, que é vazio exceto na combinação de modo *segue* com verificações de sanidade: **nenhuma comparação é emitida**, e a detecção de acesso fora dos limites é delegada a páginas não mapeadas, isto é, ao hardware.

`Wasmlifter::runClang` invoca o compilador sem qualquer `-D`, sobre um *host* x86-64, de modo que o caminho tomado é o primeiro. Isso é verificável no LLVM-IR emitido: a função de escrita gerada contém apenas `call void @i32_store_unchecked`, ao passo que, acrescentando `-DWASM_RT_MEMCHECK_BOUNDS_CHECK=1` à mesma compilação, ela passa a conter `icmp ugt` contra `mem->size` seguido de `call void @wasm_rt_trap(i32 1)`.

Portanto, na configuração de compilação realmente empregada, `Safewasm` **não é avaliado para acessos à memória linear padrão**. A condição lateral não é traduzida em código: é substituída por uma suposição sobre o *layout* de endereçamento do processo, suposição que a Seção 9.2 mostra não se sustentar aqui.

Duas dependências dessa elisão. A escolha depende do módulo e da plataforma, e convém não generalizá-la. Se o módulo tem **segmentos de dados**, o `RANGE_CHECK` da inicialização de memória é emitido de todo modo, e chamadas com o código `OOB` aparecem no IR mesmo sem `-D`. E a suposição de *host* x86-64 é implícita: o campo `targetTriple` da configuração do *lifter* é declarado e nunca consumido.

Nem toda classe de *trap* de $\mathcal{T}$ tem, necessariamente, um teste correspondente no código gerado a partir de um dado módulo. Duas causas independentes reduzem o conjunto: o módulo pode não conter a instrução que originaria o teste, um módulo sem divisão inteira não gera teste de divisão por zero, e a estratégia de verificação de limites escolhida em tempo de compilação decide se o teste de acesso é emitido ou delegado a páginas-guarda (Seção 5.3). Escrevemos por isso

$$\mathcal{T}_{\text{emit}}(M) \subseteq \mathcal{T}$$

para o subconjunto das classes cujo teste o código gerado a partir de M efetivamente contém, isto é, para as quais há uma chamada a `wasm_rt_trap` no LLVM-IR emitido. Abrevia-se

$\mathcal{T}_{\text{emit}}$ para as quais há uma chamada a `wasm_rt_trap` no LLVM-IR emitido.

A cláusula final não é ornamental, e a Seção 5.3 mostra por quê. Sob a estratégia de páginas-guarda, um *trap* de acesso fora dos limites corresponde a um programa C que *não* contém chamada a `wasm_rt_trap`: quem o sinaliza é o tratador de sinal instalado pelo *runtime*. Sem a cláusula, a equivalência seria falsa na configuração aqui analisada. E como o Map2Check não liga esse *runtime* (Seção 9.1), o segundo disjuncto não se realiza no *pipeline* real: é essa a razão de a Proposição 1 ser enunciada sobre $\mathcal{T}_{\text{emit}}$, e não sobre $\mathcal{T}$.

Esta é uma hipótese, **não um teorema demonstrado neste documento**. Assim como se assume a correção do compilador Clang, assume-se aqui a correção do WABT. Explicitá-la é importante por duas razões: toda a *soundness* da cadeia depende dela, e ela é falsificável. Uma divergência do `wasm2c`, por exemplo, apareceria como uma diferença entre a execução do `.wasm` em um *runtime* conforme e a execução do C gerado.

6 Passo 2, κ : Clang para LLVM-IR

$$\kappa : C \longrightarrow LLVM$$

implementada em `WasmLifter::runClang` e `WasmLifter::runClangEmitLLVM`:

```
clang-16 -c -emit-llvm -I<wasm-rt> -I<dir do .h> <arq>.c -o <arq>.bc
clang-16 -S -emit-llvm -I<wasm-rt> -I<dir do .h> <arq>.c -o <arq>.ll
```

A emissão do `.ll` textual não é redundante: ele é a entrada de ϵ , o passo seguinte.

Hipótese 2 (correção do Clang). κ preserva a semântica de programas C livres de comportamento indefinido.

A ressalva é especialmente relevante neste contexto. O C emitido pelo `wasm2c` é código gerado, não escrito manualmente, e emprega construções que exigem atenção à fronteira entre o modelo operacional do `wasm2c` e a semântica da linguagem C: aritmética de ponteiros sobre `data`, `setjmp/sigsetjmp` para o mecanismo de *trap* e o qualificador `__seg_gs` quando o modo *segue* está habilitado. Essa fronteira não é auditada neste documento.

7 Passo 3, ϵ : extração do ponto de entrada

$$\epsilon : LLVM_{\text{texto}} \longrightarrow Name \times \mathcal{P}(Name)$$

implementada em `WasmLifter::extractMetadata`. Ela percorre o `.ll` linha a linha com a expressão regular

```
| define.*@w2c_([A-Za-z0-9_]+)\(
```

e devolve o par (η, E) : o ponto de entrada η , reconhecido pela presença da subcadeia `_0x5Fstart` em qualquer posição da linha, não pelo sufixo, e o conjunto E das demais funções assim casadas.

Convém ser exato quanto a E , porque o texto natural engana: a expressão regular casa igualmente `define internal`, de modo que E contém **todas** as funções `w2c_` definidas, exportadas ou não. Num módulo WASI medido, 42 das 44 definições são internas.

Trata-se de **análise sintática sobre texto**, e não de **análise do módulo LLVM**. Suas condições de falha merecem ser explicitadas, pois não são meramente hipotéticas:

1. Se nenhuma função casa com `_0x5Fstart`, o caso de um módulo *reactor*, sem `_start`, que é a forma usual de um *handler* WASM de IoT, o código adota o valor padrão `w2c__start` e emite um aviso. Esse símbolo pode não existir.

2. Por construção, E e η são disjuntos. Logo um módulo cujo único ponto de interesse são funções exportadas, de novo, o caso IoT, tem η mal definido.

Portanto, ϵ é total como procedimento, mas **não é correta** para a classe de módulos que a proposta de pesquisa declara como alvo.

8 Passo 4, ω : o *harness*

O programa liftado não possui `main`. τ traduziu funções de módulo, e nenhuma delas é ponto de entrada de um processo. Define-se então

$$\omega : \text{Name} \longrightarrow \text{LLVM}$$

implementada em `Caller::generateWasmWrapperStatic`, que sintetiza literalmente:

```
int main() {
    w2c_<mod> instance;
    wasm2c_<mod>_instantiate(&instance, NULL);
    <entrada>(&instance);
    wasm2c_<mod>_free(&instance);
    return 0;
}
```

O resultado é então ligado ao módulo liftado por

$$\Lambda_0 = \text{llvm-link}(\kappa(\tau(M)), \omega(\eta)).$$

8.1 Um defeito de aridade no *wrapper*

O *wrapper* emite a chamada de instanciação com **dois** argumentos, `(&instance, NULL)`. A assinatura que o `wasm2c` gera tem um parâmetro para a instância e **um por módulo importado**: para um módulo sem importações é `void wasm2c_t_instantiate(w2c_t*)`, e a chamada com dois argumentos é erro de compilação. O *wrapper* está correto apenas para módulos com exatamente uma importação, o caso dos binários WASI, o que explica por que a validação existente compila.

O que agrava o defeito é o seu silêncio: o valor de retorno de `system()` não é verificado em `generateWasmWrapperStatic` nem na chamada de `llvm-link` que a sucede, e esta escreve a saída *sobre o próprio arquivo de entrada*. O resultado é um bitcode sem `main`, ou em estado indeterminado, produzido sem diagnóstico algum.

8.2 O que ω quantifica: nada

Formalmente, o módulo denota uma **família** de execuções, indexada pelos argumentos das funções exportadas e pelo comportamento das importações:

$$\llbracket M \rrbracket : \overline{\text{Val}} \times \text{Host} \longrightarrow \text{Outcome}.$$

O *harness* atual não instancia essa família; ele simplesmente a restringe ao caminho de execução do `_start`. A chamada $\eta(\&I)$ é nulária no que diz respeito aos argumentos do módulo: o único parâmetro explícito é o ponteiro de instância, porque η é o `_start`, cuja assinatura no WASM é `[] → []`. Nenhuma valoração ν para argumentos de funções exportadas é escolhida, pois não há chamada a essas funções no *wrapper* atual.

A consequência é precisa. Escrevendo $Reach(q)$ para o conjunto de estados alcançáveis de um programa q (a letra p fica reservada para ponteiros, na Seção 9):

$$Reach(\Lambda_0) = \bigcup_{h \in Host} Reach(\llbracket M \rrbracket(\perp, h)) \subseteq \bigcup_{\nu, h} Reach(\llbracket M \rrbracket(\nu, h)),$$

e a inclusão é **estrita** sempre que o módulo possui alguma função exportada com aridade positiva. O símbolo $\perp$ não denota um valor: denota a ausência da chamada.

Alocação. `wasm_rt_allocate_memory` troca o `mmap` do WABT por um **único `calloc`**, com a justificativa, registrada no arquivo, de que o KLEE modela mal `mmap`. Isso confirma, de forma mais direta do que a análise do WABT permitiria, a premissa da Proposição 2: a memória linear é uma alocação de *heap* única.

Sinalização de *trap*. A implementação do WABT executa `WASM_RT_LONGJMP(g_wasm_rt_jump_buf, code)`, isto é, desvio não local para um *buffer* que só existe se o *embedder* tiver estabelecido um `wasm_rt_impl_try`, o que o *wrapper* não faz. O *stub* substitui esse mecanismo por chamada direta a `map2check_error()`. **É esse o canal pelo qual um *trap* do WASM se torna violação observável**, e não um `abort` genérico. O código do *trap* é descartado: todas as classes de $\mathcal{T}$ colapsam em um sinal indistinguível.

Inicialização. O `wasm2c` emite `assert(wasm_rt_is_initialized())` no início de `instantiate`, e o *wrapper* nunca chama `wasm_rt_init()`. Com o *runtime* do WABT isso abortaria antes de qualquer execução do módulo. O *stub* evita o problema declarando a *flag* verdadeira de saída, propriedade que depende de `wasm_rt_free()` não ser chamado, e o *wrapper* chama `wasm2c_<mod>_free`, que é outra função.

9.2 A incompatibilidade entre compilação e *runtime*

Combinando as Seções 5.3 e 9.1, obtém-se uma inconsistência que nenhuma das duas revela isoladamente:

O código gerado **omite** a verificação de limites porque a compilação supõe que a memória linear está cercada por páginas-guarda; o *runtime* efetivamente ligado aloca essa memória com `calloc`, que **não tem páginas-guarda**.

Um acesso fora de $[0, \ell)$ não é, portanto, detido por verificação explícita, ela não foi emitida, nem por hardware, não há região protegida. É um transbordamento de *heap* sobre o bloco do `calloc`.

9.3 O mapa de objetos existe, e não dispara

Seria natural supor que a instrumentação do Map2Check nada sabe sobre a estrutura interna da memória linear. **Não é o caso**, e o fato altera substancialmente o plano de trabalho que a Seção 13 deriva.

O `MemoryTrackPass` possui um caminho WASM dedicado, ativado pelo *flag* `-wasm-mode` que `Caller::callPass` passa no modo WASM: funções cujo nome começa em `w2c_` e termina em `_malloc` ou `_dlmalloc`, o alocador *do módulo*, são reconhecidas e desviadas para `map2check_wasm_malloc(offset, size)`, cujos argumentos são **deslocamentos dentro da memória linear**, não ponteiros do processo. O *library-backend* grava-os no `AllocationLog`, e `map2check_wasm_check_access` reporta `FALSE_OVERFLOW` quando um acesso não cai em alocação registrada.

Isso é, em substância, o mapa de objetos *O* da Seção 13, implementado para as alocações de *heap* do módulo.

Por que não dispara. Duas razões, ambas verificadas no IR produzido nas condições de `runClang`:

1. A instrumentação de acesso exige um `GetElementPtrInst` com ao menos três operandos. O acesso à memória linear no IR emitido é `getelementptr inbounds i8, ptr %11, i64 %12, dois` operandos. A função retorna sem instrumentar.
2. Sem `-O`, o acesso sequer é um `StoreInst` ou `LoadInst`: `i32_store_unchecked` termina em `call void @llvm.memcpy.p0.p0.i64`. O laço que percorre acessos nunca o alcança.

Há ainda um obstáculo de projeto, e não apenas de casamento de padrão: se a instrumentação disparasse, ela reportaria `FALSE_OVERFLOW` para todo acesso a **dado estático**, que nunca entra no `AllocationLog`, por não provir do alocador do módulo. O mapa cobre o *heap* do módulo, não os objetos estáticos do contraexemplo da Seção 11.

A cadeia completa é

$$\Lambda = \pi \circ \text{link}(\omega \circ \text{pr}_1 \circ \epsilon, \cdot) \circ \kappa \circ \tau$$

seguida do motor de geração de entradas, KLEE ou LibFuzzer, sobre $\Lambda(M)$.

10 Proposição 1: o que a cadeia preserva

Proposição 1 (*soundness* para alcançabilidade de *trap*). Sob as Hipóteses 1 e 2, se a execução de $\Lambda(M)$ alcança `wasm_rt_trap`, então a execução correspondente de M reduz a *trap*.

Demonstração (esboço). Pela Hipótese 2, a execução de Λ_0 corresponde à do C gerado. Pela Hipótese 1, alcançar `wasm_rt_trap` no C corresponde a alcançar *trap* no WASM. A transformação π acrescenta código que opera sobre um estado sombra disjunto do estado do programa, logo preserva alcançabilidade. ■

Uma condicional que a demonstração carrega. A Hipótese 2 vale para programas C livres de comportamento indefinido, e a Seção 6 registra que essa fronteira não foi auditada para o código que o `wasm2c` emite. A proposição é, portanto, condicional a ela: vale *desde que* o C gerado esteja livre de comportamento indefinido. Auditar essa fronteira, ou compilar com sanitizadores para exercitá-la, torna a proposição incondicional; enquanto não se faça, a condicional deve acompanhar qualquer enunciado do resultado.

10.1 Leitura correta do resultado

O resultado é real e é útil, mas o seu escopo é mais estreito do que a formulação ingênua sugeriria, e convém enunciá-lo sem folga.

O que a proposição dá. Ausência de falso positivo: um `wasm_rt_trap` alcançado corresponde a um *trap* genuíno de M .

O que ela não dá. *Detectabilidade*, que é a recíproca. Para concluir que M *trapa* implica $\Lambda(M)$ sinalizar, seriam necessárias duas condições adicionais: que a classe pertença a $\mathcal{T}_{\text{emit}}(M)$, e que o *harness* alcance o teste, e a Seção 8.2 demonstra que $\text{Reach}(\Lambda_0)$ está estritamente contido nas execuções de M . Conter uma chamada a `wasm_rt_trap` não é o mesmo que alcançá-la.

Quais classes, concretamente. Determinamos $\mathcal{T}_{\text{emit}}$ por inspeção do IR, contando as chamadas a `wasm_rt_trap` por código. Para um módulo com divisão inteira, acesso indexado e instrução inalcançável, e sem segmentos de dados, o IR contém exatamente três chamadas, com os códigos correspondentes a `INT_OVERFLOW`, `DIV_BY_ZERO` e `UNREACHABLE`, e **nenhuma** com o código de `OoB`. Este é o conjunto estabelecido por observação. As demais classes cujo teste não depende da estratégia de limites, `INVALID_CONVERSION`, `CALL_INDIRECT`, `NULL_REF`, provêm dos mesmos macros incondicionais e devem, por inferência sobre a estrutura do tradutor, pertencer também a $\mathcal{T}_{\text{emit}}$; nenhum experimento aqui descreito as exercitou.

Como o sinal chega. Não por `abort`. A implementação do WABT executa um `longjmp`; o `stub` do `Map2Check` (Seção 9.1) a substitui por chamada a `map2check_error()`, que é o sinal consumido pelo modo de alcançabilidade. Como o `stub` descarta o código do `trap`, as classes chegam ao verificador indistinguíveis entre si: a cadeia responde “houve *trap*”, nunca “houve divisão por zero”.

O efeito de uma *flag*. Compilar com `-DWASM_RT_MEMCHECK_BOUNDS_CHECK=1` acrescenta `OoB` a $\mathcal{T}_{\text{emit}}$ e elimina a incompatibilidade da Seção 9.2. Acrescenta também `EXHAUSTION`, e por um caminho indireto: desligar páginas-guarda desliga o tratador de esgotamento de pilha, e o cabeçalho então liga a contagem de profundidade, de modo que o código gerado passa a emitir o teste em cada função. Não produz, contudo, a igualdade $\mathcal{T}_{\text{emit}} = \mathcal{T}$: `UNCAUGHT_EXCEPTION` e `UNALIGNED` dependem de tratamento de exceções e de atômicos sobre memória compartilhada, que a *flag* não habilita.

11 Proposição 2: o que a cadeia não preserva

Antes do enunciado é preciso dizer o que se entende por *objeto*, pois a noção não pertence ao WebAssembly e é justamente a que falta. Chamamos objeto cada intervalo de $[0, \ell)$ que o programa-fonte tratava como uma variável ou alocação distinta, e que a compilação para WASM não representa: a Seção 13 formaliza isso como um mapa $O : [0, \ell) \rightarrow \text{ObjID}$. Um acesso *fora do objeto pretendido* é um acesso cujo intervalo não está contido em um único objeto.

Proposição 2 (incompletude para segurança de memória). Existem um módulo M , uma execução e um acesso nela tais que esse acesso **sobrescreve um objeto que não é o pretendido** (uma violação genuína de CWE-787) e, ainda assim, nenhum acesso da execução viola `SafeC`, e a execução não alcança `wasm_rt_trap`.

Demonstração (por construção). Tome o módulo compilado a partir de

```
int buf[4];
char token[16];
int f(int i) { buf[i] = 7; return buf[0]; }
```

com $i = 4$. Sejam a_{buf} e a_{token} os deslocamentos dos dois objetos na memória linear.

A adjacência entre eles não é estipulada: é observada. Compilando esta fonte para `wasm32`, obtêm-se $a_{\text{buf}} = 1024$ e $a_{\text{token}} = 1040$, ambos os objetos carecem de inicializador, logo residem na mesma seção, e a ordem relativa segue a ordem de declaração. **A ressalva pertinente é sobre a seção de alocação, não sobre o linker:** dar um inicializador não nulo a apenas um deles separa-os em seções distintas e pode inverter a ordem, caso em que o índice que atinge o vizinho passa a ser negativo. O argumento não depende da direção.

No WASM. O acesso tem endereço efetivo $ea = a_{\text{buf}} + 16 = a_{\text{token}}$. Como $ea + 4 \leq \ell$, a memória linear tem ao menos uma página, logo $\ell \geq 65\,536$, e os objetos estão no início dela, a

condição lateral da regra `LOAD` é *satisfeita*. A redução não produz `trap`: produz uma escrita bem definida. O *token* foi sobrescrito e a execução prossegue normalmente.

No C gerado. Sob `BOUND_CHECK`, o teste `res > mem->size` equivale a $ea + 4 > \ell$, que é falso. Sob `GUARD_PAGES`, a configuração real, Seção 5.3, não há sequer teste a avaliar. Em qualquer das duas, nenhum *trap*.

Sob a instrumentação. Como visto, `data` é uma *única* alocação cobrindo $[0, \ell)$. Logo Σ contém o registro $(data, \ell, true)$ e, para $p = data + ea$ e $n = 4$, valem $data \leq p$ e $p + 4 \leq data + \ell$. Portanto $Safe_C(\Sigma, p, 4)$ é verdadeiro e nenhuma violação é reportada. A instrumentação WASM dedicada da Seção 9.3 tampouco alcança o caso: além de não disparar sobre a forma do IR gerado, ela cobre alocações do *heap* do módulo, e *buf* e *token* são objetos estáticos.

Execução. O programa correspondente, ligado ao *runtime* do WABT, foi executado. Com $i = 4$, o primeiro byte de *token* passa de 0 a $119 = 0x77$, *token* é vetor de `char`, logo o vizinho é atingido byte a byte, e o processo termina com código de saída 0, sem *trap*. Para comparação, um índice bem fora de $[0, \ell)$ termina com código 134, mas por via do tratador de sinal do *runtime* do WABT, que o Map2Check não liga. ■

O ponto não é uma limitação específica do modo `memtrack`. O problema é que τ **não preserva a informação necessária**: a partição da memória em objetos existe no código C original, mas é descartada durante a compilação para WASM, porque o modelo de memória do WebAssembly não representa essas fronteiras como objetos distintos. O que chega ao Map2Check é um vetor de bytes com uma única fronteira relevante: a fronteira externa da memória linear.

Convém não converter isso num efeito de grau. A invisibilidade não depende do tamanho da memória: ela decorre de a violação ser intra-memória, e vale para *qualquer* ℓ . Já numa memória de uma única página, o menor valor possível, $\ell = 65\,536$, um acesso que ultrapasse o limite de um objeto por poucos bytes permanece a dezenas de milhares de bytes da única fronteira que o modelo preserva. O tamanho apenas amplia a região em que isso ocorre; não é a variável explicativa.

12 Consequência para a validação existente

O resultado acima muda a forma como deve ser interpretado `docs/reports/wasm-juliet-validation.csv`. Todos os casos ali registrados retornam `FALSE`, isto é, um defeito foi encontrado. Contudo, as Proposições 1 e 2, consideradas em conjunto, mostram que há **dois mecanismos possíveis** por trás desse veredito:

- (a) o *overflow* foi grande o bastante para sair de $[0, \ell)$, o `RANGE_CHECK` do WABT disparou, e o que foi detectado foi a verificação do WABT; ou
- (b) a instrumentação do Map2Check detectou algo por conta própria.

A planilha **não distingue esses dois casos**, e a Proposição 2 estabelece que (b) é impossível para violações intra-memória. Portanto, enquanto essa separação não for feita, a validação WASM não sustenta, por si só, a alegação de que o *Map2Check* detectou os defeitos. Os mesmos resultados são igualmente compatíveis com a hipótese de que o WABT detectou os acessos inválidos e o Map2Check apenas reportou o *abort* resultante.

A separação é simples: basta instrumentar `wasm_rt_trap` para registrar qual caminho produziu cada *abort*. Essa deve ser a primeira medição realizada na frente WASM, pois ela precede qualquer nova alegação sobre capacidade de detecção.

13 O que a modelagem de memória linear tem de acrescentar

A proposta de pesquisa prevê uma fase de “modelagem da memória linear”. Nos termos deste documento, o que falta é exatamente um objeto: um **mapa de objetos**

$$O : [0, \ell) \rightarrow \mathbf{ObjID},$$

com o predicado refinado

$$\text{Safe}_{\text{lin}}(\mu, O, ea, N) \equiv \text{Safe}_{\text{wasm}}(\mu, ea, N) \wedge$$

$$\exists o \in \mathbf{ObjID} : \forall k \in [ea, ea + N/8) : k \in \text{dom}(O) \wedge O(k) = o.$$

Em palavras: além de estar dentro dos limites da memória linear, o acesso inteiro precisa estar **contido em um único objeto**. Com Safe_{lin} no lugar de Safe_{C} , o contraexemplo da Proposição 2 passa a ser detectável, pois $O(a_{\text{buf}} + 16) = \text{token} \neq \text{buf}$.

Isso torna preciso o requisito sobre o *lifter*. O mapa O **não pode ser recuperado integralmente a partir do .wasm** se a informação de objetos não tiver sido preservada em alguma forma auxiliar. As três fontes possíveis são:

1. s2 5880(.)–329asem O

Passo	Função	Implementação	Preserva	Perde
0	M	entrada <code>.wasm</code>		o <i>layout</i> de objetos, já perdido na compilação <i>para</i> WASM
1	τ	<code>wasm2c</code> , WABT 1.0.41	semântica, bytes da memória, os <i>traps</i> de $\mathcal{T}_{\text{emit}}$	nada além do que já faltava
2	κ	<code>clang-16 -emit-llvm</code>	semântica de programas C livres de comportamento indefinido	o teste de limites, elidido ao supor páginas-guarda
3	ϵ	regex sobre o <code>.ll</code>	ponto de entrada, quando há <code>_start</code>	módulos <i>reactor</i> , o caso IoT
4	ω	<i>wrapper</i> e <code>llvm-link</code>	a execução de <code>_start</code>	toda função exportada de aridade positiva, e o <i>host</i>
5	π	passes do Map2Check, mais os <i>stubs</i> de <code>wasm_rt_*</code>	alcançabilidade	a classe do <i>trap</i> , descartada pelo <i>stub</i>

3. **Separar as vias de detecção.** Fazer o *stub* de `wasm_rt_trap` registrar o código recebido, e o classificador registrar por qual via o veredito chegou. Reprocessar em seguida a validação Juliet-WASM. Nenhuma alegação sobre a frente WASM se sustenta antes disso.
4. **Corrigir a aridade do *wrapper* e verificar os retornos.** O defeito da Seção 8.1 restringe a ferramenta a módulos com exatamente uma importação, e falha em silêncio fora dessa classe.
5. **Enunciar a Proposição 1 como resultado.** A *soundness* para $\mathcal{T}_{\text{emit}}$ já existe na implementação atual e não está sendo reivindicada. É publicável por si, e independente da modelagem de memória.
6. **Corrigir ϵ e ω para módulos *reactor*.** Gerar uma chamada por função *exportada*, e não por toda função de E , que inclui as internas, com `__VERIFIER_nondet_<t>()` em cada parâmetro. Sem isso, o alvo declarado da proposta permanece fora do alcance da ferramenta, por um motivo puramente mecânico.
7. **Estender o mapa de objetos aos dados estáticos.** O mapa existente cobre o *heap* do módulo; o contraexemplo da Seção 11 usa objetos estáticos, que nunca entram no `AllocationLog`. Esta é a única das sete que constitui pesquisa.

A Protocolo de verificação

As afirmações deste documento sobre o código gerado e sobre o comportamento em execução não foram inferidas da documentação das ferramentas: foram verificadas. Registram-se aqui os procedimentos, para que possam ser repetidos e contestados.

Ambiente: imagem `ghcr.io/hbgit/map2check-dev`, com WABT 1.0.41 e `clang-16`. Preparação comum a todos os experimentos:

```
clang-16 --target=wasm32 -nostdlib -Wl,--no-entry -Wl,--export-all \
    -O1 -o t.wasm t.c
wasm2c t.wasm -o gen.c
```

A.1 A elisão do teste de limites

Compilar a saída do `wasm2c` exatamente como `WasmLifter::runClang` o faz, e inspecionar a função de escrita no IR:

```
| clang-16 -S -emit-llvm -I$WABT/include -I. gen.c -o gen.ll  
| awk '/define.*i32_store_default32/,/^}/' gen.ll
```

O corpo contém apenas `call void @i32_store_unchecked`. Repetindo com `-DWASM_RT_MEMCHECK_BOUNDS_CHECK=1`, passa a conter `icmp ugt contra mem->size` e `call void @wasm_rt_trap(i32 1)`.

A.2 Determinação de $\mathcal{T}_{\text{emit}}$

Contagem das chamadas por código de *trap*:

```
| grep -o '@wasm_rt_trap(i32 noundef [0-9]*)' gen.ll | sort | uniq -c
```

Para um módulo com divisão inteira, acesso indexado e instrução inalcançável, e **sem segmentos de dados**, observam-se os códigos 2, 3 e 5, e nenhum código 1. Pela ordem do `enum wasm_rt_trap_t` em `wasm-rt.h`, esses códigos são `INT_OVERFLOW`, `DIV_BY_ZERO` e `UNREACHABLE`, e o ausente é `OOB`.

Dando **inicializadores não nulos** aos globais, de modo que o módulo passe a ter segmento de dados, o mesmo `wasm2c` sem `-D` emite duas chamadas com o código 1, provenientes do `RANGE_CHECK` na inicialização de memória. Daí a dependência de $\mathcal{T}_{\text{emit}}$ em relação ao módulo, e não apenas à estratégia de verificação.

Acrescentando `-DWASM_RT_MEMCHECK_BOUNDS_CHECK=1`, surgem ainda oito chamadas com o código de `EXHAUSTION`, uma por função: desligar páginas-guarda desliga o tratador de esgotamento de pilha, e o cabeçalho passa a ligar a contagem de profundidade.

A.3 O contraexemplo

O módulo exporta funções que devolvem os deslocamentos de `buf` e `token` na memória linear, de modo que o *layout* escolhido seja observado em vez de suposto. Ligando ao *runtime* do WABT:

```
| clang-16 -O0 -I$WABT/include -I. h.c gen.c \  
| $WABT/share/wabt/wasm2c/wasm-rt-impl.c \  
| $WABT/share/wabt/wasm2c/wasm-rt-mem-impl.c -o prog
```

Com a fonte da Seção 11, em que ambos os objetos carecem de inicializador e portanto residem na mesma seção, obtém-se $a_{\text{buf}} = 1024$ e $a_{\text{token}} = 1040$; o índice $i = 4$ leva o primeiro byte de `token` de 0 a $119 = 0x77$, e o processo termina com código de saída 0. Um índice bem fora de $[0, \ell)$ termina com código 134.

Dar inicializador não nulo a apenas um dos objetos separa-os em seções distintas e pode inverter a ordem relativa, caso em que o índice que atinge o vizinho passa a ser negativo. O argumento da Proposição 2 não depende da direção.

A.4 A aridade do *wrapper*

Compilar um *harness* que reproduza a chamada emitida por `generateWasmWrapperStatic` contra um módulo sem importações: o `clang` responde `too many arguments to function call, expected 1, have 2`. A assinatura gerada confere-se com

```
| grep -nE '^void wasm2c.*_instantiate' gen.c
```

A.5 O que não foi verificado

Os procedimentos acima exercitam a cadeia até o LLVM-IR e o comportamento do *runtime* do WABT. $\Lambda(M)$ **nunca foi executado de ponta a ponta**: isso exigiria um *build* completo do Map2Check, e todas as observações de execução aqui registradas foram feitas sobre o *runtime* do WABT, que o *pipeline* não liga (Seção 9.1).

Em consequência, três afirmações do documento permanecem derivadas de leitura de código e não de execução observada: a de que Σ recebe a memória linear como alocação única; a de que a instrumentação WASM dedicada não dispara sobre o IR do `wasm2c`; e a de que, sob os *stubs*, um acesso muito além de $[0, \ell)$ não produz sinal algum. As três são verificáveis assim que houver um *build* disponível, e são as primeiras que se deveria fechar.

B Glossário de símbolos e termos

Reúne todo símbolo empregado no documento. Para cada um: a leitura em palavras, a definição, e a seção em que é introduzido. Os grupos seguem a ordem da cadeia.

B.1 Notação de base

Símbolo	Leitura	Definição
$\mathbb{B}$	“o conjunto dos bytes”	$\{0, \dots, 255\}$.
$\mathbb{N}$	“os naturais”	Usado apenas para o número de páginas.
$[a, b)$	“intervalo semiaberto”	Inteiros k com $a \leq k < b$; inclui a , exclui b .
$\text{dom}(f)$	“domínio de f ”	Conjunto dos pontos em que a função parcial f está definida.
$\mathcal{P}(X)$	“partes de X ”	Conjunto de todos os subconjuntos de X .
$\rightharpoonup$	“função <i>parcial</i> ”	$f : A \rightharpoonup B$ pode não estar definida em todo A .
$\rightarrow$	“função total”	Contrasta com o anterior; definida em todo o domínio.
P	“tamanho de página”	A constante 65 536, fixada pela especificação do WASM.

B.2 O módulo e seu estado de execução

Símbolo	Leitura	Definição
M	“o módulo”	O binário <code>.wasm</code> de entrada, visto como tupla de dez componentes.
$types$ etc.		Os componentes de M : assinaturas, funções, tabelas, memórias, globais, segmentos de elementos e de dados, função de início, importações e exportações.
$mems$	“as memórias”	Componente de cardinalidade no máximo 1 na versão 1.0, a hipótese estrutural de todo o documento.
$exports$	“as exportações”	$Name \rightarrow ExternIdx$: nomes visíveis de fora do módulo.
$imports$	“as importações”	Funções que o <i>host</i> fornece; em IoT, o acesso a sensores, rede e banco.
$Name$	“nomes”	Cadeias que identificam funções exportadas ou importadas.
$ExternIdx$	“índice externo”	Referência a uma entidade do módulo (função, tabela, memória, global).
$valtype$	“tipos de valor”	$\{i32, i64, f32, f64\}$.
Val	“valores”	Conjunto dos valores desses tipos.
s	“o <i>store</i> ”	Estado global: funções, tabelas, memórias e globais instanciadas.
F	“o <i>frame</i> ”	Estado local do quadro corrente: variáveis locais e a instância ativa.
e^*	“a pilha de instruções”	Sequência de instruções ainda por executar.
$s; F; e^*$	“uma configuração”	O estado completo de execução do WASM.
$\hookrightarrow$	“reduz em um passo”	Relação de redução em passos pequenos entre configurações.
$trap$	“ <i>trap</i> ”	Termo distinguido e absorvente: nenhuma regra o reduz. É como o WASM sinaliza erro irrecuperável.
$\mathcal{T}$	“as classes de <i>trap</i> ”	As dez razões que o WABT distingue (OOB, divisão por zero, ...).
$\mathcal{T}_{emit}(M)$	“as classes emitidas”	Subconjunto de $\mathcal{T}$ cujo teste o código gerado a partir de M de fato contém.

B.3 A memória linear e o acesso

Símbolo	Leitura	Definição
μ	“a instância de memória”	O par (d, ℓ) .
d	“os bytes”	$d : [0, \ell) \rightarrow \mathbb{B}$: o conteúdo da memória linear.
ℓ	“o tamanho”	Tamanho da memória linear em bytes; sempre múltiplo de P .
n	“páginas”	Número de páginas, com $\ell = n \cdot P$. (Não confundir com o n de <code>SafeC</code> .)
t	“o tipo do acesso”	Elemento de <i>valtype</i> ; prefixo em <code>t.load</code> .
N	“largura em bits”	8, 16, 32 ou 64. O acesso toca $N/8$ bytes.
i	“índice dinâmico”	Valor <code>i32</code> vindo da pilha de operandos.
<code>o set</code>	“deslocamento estático”	Constante imediata da instrução.
ea	“endereço efetivo”	$ea = i + \text{o set}$. Índice em bytes <i>dentro</i> da memória linear, um inteiro, não um ponteiro do processo.
c	“o valor lido”	Resultado da leitura, de tipo t .
a_{buf}, a_{token}	“deslocamentos”	Posições dos dois objetos do contraexemplo dentro da memória linear.

B.4 As funções da cadeia

Símbolo	Leitura	Definição
τ	“ <i>tau</i> , o tradutor”	$Wasm \rightarrow C$. O <code>wasm2c</code> do WABT.
κ	“ <i>kappa</i> , o compilador”	$C \rightarrow LLVM$. O <code>clang-16</code> .
ϵ	“ <i>epsilon</i> , a extração”	$LLVM_{\text{texto}} \rightarrow Name \times \mathcal{P}(Name)$. Descobre o ponto de entrada.
ω	“ <i>ômega</i> , o <i>harness</i> ”	$Name \rightarrow LLVM$. Sintetiza o <code>main</code> que falta.
π	“ <i>pi</i> , a instrumentação”	Os passes do <code>Map2Check</code> ; o par $(\pi_{\text{code}}, \Sigma)$.
Λ	“ <i>lambda</i> , a cadeia”	A composição completa das cinco funções.
Λ_0	“a cadeia sem instrumentar”	<code>llvm-link</code> ($\kappa(\tau(M)), \omega(\eta)$): já ligado, ainda não instrumentado.
η	“ <i>eta</i> , o ponto de entrada”	Nome da função de entrada detectada por ϵ .
E	“as demais funções”	Segundo componente devolvido por ϵ .
$Wasm, C, LLVM$		Os conjuntos de programas de cada linguagem, domínios e contradomínios das funções acima.
mod	“o módulo, no nome”	Prefixo do <i>mangling</i> <code>w2c_<mod>_<função></code> .
f	“uma função do módulo”	Função de M , cuja tradução $\tau(f)$ é uma função C .
I	“a estrutura de instância”	O registro C em que τ realiza o <i>store</i> s ; o <i>wrapper</i> declara uma e passa <code>&I</code> .
$\approx$	“corresponde a”	Relação entre configuração WASM e estado C : memória igual byte a byte, globais e locais de mesmo valor.
σ, σ'	“estados C ”	Estados do programa C gerado.
$\rightarrow^+$	“reduz em um ou mais passos”	Fecho transitivo da redução no lado C .

B.5 Semântica denotacional e alcançabilidade

Símbolo	Leitura	Definição
$\llbracket M \rrbracket$	“o significado de M ”	$\overline{Val} \times Host \rightarrow Outcome$: a <i>família</i> de execuções do módulo.
$\overline{Val}$	“uma tupla de valores”	Os argumentos das funções exportadas.
$Host$	“o ambiente”	O comportamento das funções importadas.
$Outcome$	“o desfecho”	Resultado de uma execução.
ν	“ <i>nu</i> , uma valoração”	Uma escolha concreta de argumentos em $\overline{Val}$.
h	“um <i>host</i> ”	Uma escolha concreta em $Host$.
$\perp$	“ <i>bottom</i> ”	Não é um valor: denota a <i>ausência</i> da chamada, quando o <i>harness</i> não invoca a função.
$Reach(q)$	“os alcançáveis de q ”	Conjunto de estados alcançáveis do programa q . A letra q , e não p , para não colidir com o ponteiro de <code>SafeC</code> .

B.6 A instrumentação e os predicados de segurança

Símbolo	Leitura	Definição
π_{code}	“a parte que insere código”	Componente de π que acrescenta as verificações.
Σ	“ <i>sigma</i> , o estado sombra”	Conjunto das alocações vivas mantido em tempo de execução pelo <code>-memtrack</code> .
base, b	“o início”	Endereço inicial de uma alocação registrada.
size, z	“o tamanho”	Tamanho em bytes dessa alocação.
alive	“viva”	Verdadeiro enquanto a alocação não foi liberada.
p	“o ponteiro”	Endereço C do acesso sendo verificado. Diferente de ea : p é um ponteiro do processo, ea é um índice dentro da memória linear.
n	“bytes do acesso”	Largura do acesso em Safe_C . (Não confundir com o n de páginas.)
$\text{Safe}_{\text{wasm}}(\mu, ea, N)$	“seguro para o WASM”	$ea + N/8 \leq \ell$: o acesso cabe na memória linear.
$\text{Safe}_C(\Sigma, p, n)$	“seguro para o C”	Existe em Σ uma alocação viva que contém $[p, p + n)$.
$\text{Safe}_{\text{lin}}(O, ea, N)$	“seguro na memória linear”	O predicado refinado: além de caber na memória, o acesso cabe <i>dentro de um único objeto</i> .
O	“o mapa de objetos”	$[0, \ell) \rightarrow \text{ObjID}$: a que objeto pertence cada byte. É a informação que o <i>lifting</i> apaga.
ObjID	“identificador de objeto”	Contradomínio de O ; distingue <code>buf</code> de <code>token</code> .
o	“um objeto”	Elemento de ObjID .
k	“um byte do acesso”	Índice que percorre $[ea, ea + N/8)$ na definição de Safe_{lin} .

B.7 Três distinções que o glossário não deve deixar passar

1. **ea contra p .** ea é um inteiro em $[0, \ell)$, uma posição *dentro* da memória linear, como o WASM a enxerga. p é um ponteiro do processo hospedeiro, e a relação entre os dois é $p = \text{mem} \rightarrow \text{data} + ea$.
Essa mudança de referencial é, porém, **a parte inócua**: sendo a memória linear uma alocação única, $\text{Safe}_C(\Sigma, \text{data} + ea, n)$ e $\text{Safe}_{\text{wasm}}(\mu, ea, N)$ são *equivalentes*: uma é a translação da outra. O que separaria os dois predicados seria uma partição em objetos, que nenhum dos dois possui. É esse o conteúdo da Proposição 2, e não a troca de coordenadas.
2. **Os dois n .** Em $\ell = n \cdot P$, n conta *páginas*; em $\text{Safe}_C(\Sigma, p, n)$, n conta *bytes do acesso*. A colisão é herdada das convenções das duas comunidades e está registrada aqui por não ser resolvida no texto.
3. **$\mathcal{T}$ contra $\mathcal{T}_{\text{emit}}$.** O primeiro é o que o WebAssembly *define* como *trap*; o segundo é o que o código gerado, para um dado módulo e uma dada configuração de compilação, efetivamente *testa*. A distância entre os dois é o assunto da Seção 5.3.